

Project Design Phase-II

Technology Stack (Architecture & Stack)

Date	01 October 2026
Team ID	SWTID-2026-3408
Project Name	Streaming IT Procurement: Automating Standard Laptop Orders
Maximum Marks	4 Marks

Technical Architecture:

Architecture of the automated standard laptop ordering system: users access a web portal, cloud-hosted application logic validates, approves and places orders, data is held in managed storage, and external services handle login, vendor ordering and notifications.

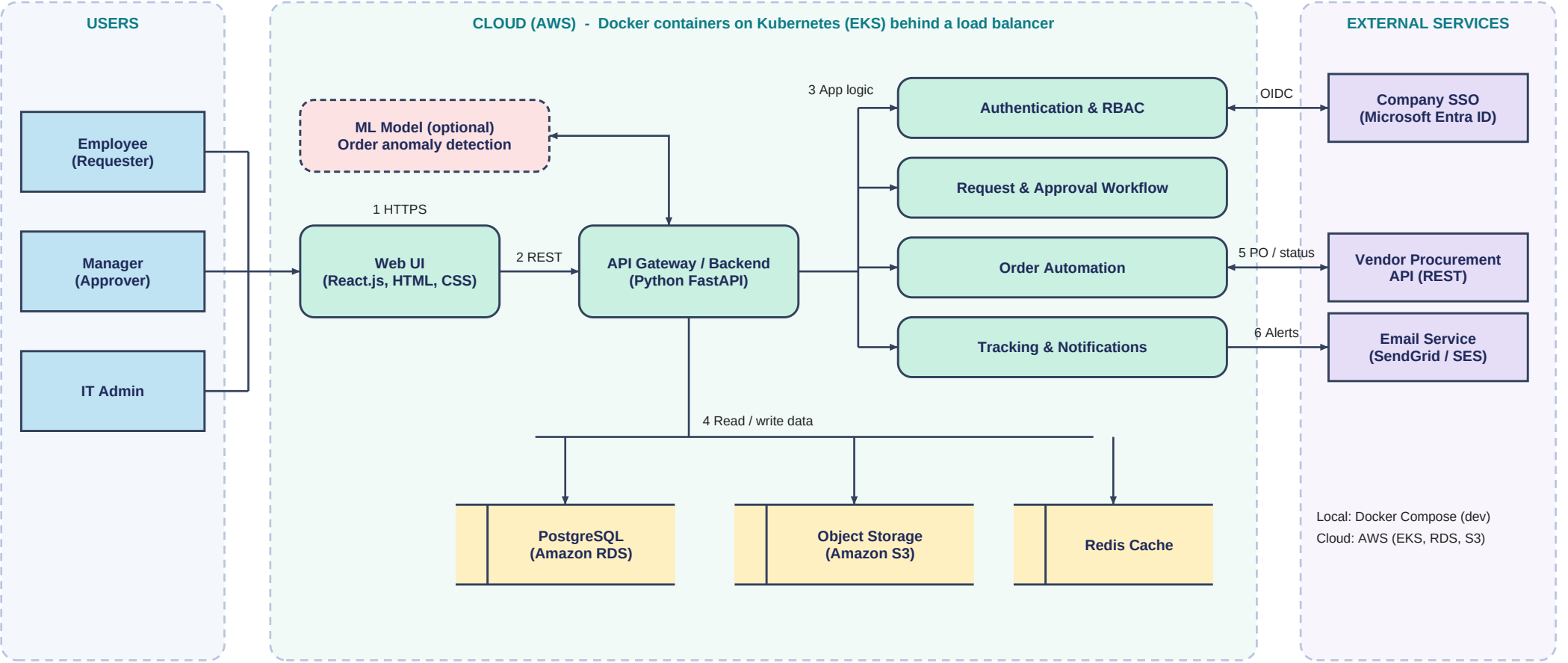


Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1.	User Interface	Responsive web portal where employees request laptops, managers approve and IT admins manage the catalog and orders.	HTML, CSS, JavaScript, React.js
2.	Application Logic-1	Authentication and role-based access control (Employee, Manager, IT Admin).	Python (FastAPI), OAuth 2.0 / OpenID Connect
3.	Application Logic-2	Request validation against the approved catalog and manager approval workflow with budget-limit checks.	Python (FastAPI), rules engine
4.	Application Logic-3	Automated purchase order generation and order placement with the vendor.	Python, Requests (REST client)
5.	Application Logic-4	Order tracking, status notifications and asset registration after delivery.	Python, Celery background workers
6.	Database	Stores users, roles, laptop catalog, requests, approvals, orders and asset inventory (relational data).	PostgreSQL
7.	Cloud Database	Managed database service with automated backups and high availability.	Amazon RDS (PostgreSQL)
8.	File Storage	Stores purchase orders, vendor invoices and delivery documents.	Amazon S3
9.	External API-1	Vendor procurement API used to place orders and receive shipment and invoice updates.	Vendor REST API (e.g. Dell / HP / Lenovo)
10.	External API-2	Company single sign-on for secure employee login.	Microsoft Entra ID (OAuth 2.0 / OIDC)
11.	External API-3	Sends email notifications for request, approval and delivery updates.	SendGrid / Amazon SES
12.	Machine Learning Model	Optional model to flag unusual order quantities or duplicate requests.	scikit-learn (Isolation Forest)
13.	Infrastructure (Server / Cloud)	Application deployed in containers on the cloud. Local Server Configuration: Docker Compose for development. Cloud Server Configuration: AWS EKS cluster with auto-scaling worker nodes.	Docker, Kubernetes (Amazon EKS), AWS

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1.	Open-Source Frameworks	Open-source frameworks used to build the front end, back end, data layer and deployment.	React.js, FastAPI, PostgreSQL, Redis, Celery, Docker, Kubernetes
2.	Security Implementations	Single sign-on, role-based access control, encrypted traffic and data, hashed credentials and secrets, input validation, audit log of approvals and orders, web application firewall.	OAuth 2.0 / OIDC, IAM roles, TLS 1.2+, AES-256, SHA-256, OWASP Top 10, AWS WAF
3.	Scalable Architecture	3-tier design (UI, application, database). Stateless containers scale horizontally, and vendor calls run through a background queue so spikes such as bulk onboarding orders do not slow the portal.	3-tier, Docker, Kubernetes auto-scaling, Celery queue
4.	Availability	Load balancer spreads traffic across multiple containers and zones; the managed database runs multi-zone with automated backups; failed containers restart automatically.	AWS Application Load Balancer, Multi-AZ RDS, Kubernetes self-healing
5.	Performance	Catalog and session data cached, paginated queries, heavy tasks (PO generation, emails) handled asynchronously, static UI files served through a CDN.	Redis cache, Amazon CloudFront CDN, Celery